SECURITY

Security Architecture & Assurance Report

How TecresTrackIT AMS protects every account, session and organization: the architecture, the controls built into it, and the penetration testing that verified them.

RESTRICTED: TECRES AND CUSTOMERS UNDER NDA

TecresTrackIT AMS
Version 1.0 · 26 September 2026

...go beyond paper
Tecres

Document control

Document	Security Architecture & Assurance Report
Version	1.0
Date	26 September 2026
Audience	Tecres leadership and engineering; customer IT, security and procurement teams
Systems covered	TecresTrackIT AMS backend API and web application (backend commit `cef8df1` with the 26 September 2026 token hardening; frontend commit `1c21661`)
Assurance activities	Architecture and source code review; automated penetration test suite; live black-box testing; dependency audit
Related documents	Technical Documentation (engineering reference); Security Remediation Register (internal)

Contents

1 Executive summary — ALL READERS

2 Security architecture at a glance — ALL READERS

3 Identity and authentication — SECURITY, ENGINEERING

4 Session management — SECURITY, ENGINEERING

5 Authorization and access control — SECURITY, ENGINEERING

6 Tenant isolation — SECURITY, ENGINEERING

7 Data protection — SECURITY, ENGINEERING

8 Application and API hardening — SECURITY, ENGINEERING

9 Monitoring, audit trail and response — ALL READERS

10 Secure development practices — ENGINEERING

11 Penetration testing — ALL READERS

A Re-running the security tests — ENGINEERING

B Glossary — ALL READERS

ALL READERS

1 Executive summary

TecresTrackIT AMS was designed security-first. It is a from-scratch platform built specifically to prevent the classes of weakness most common in multi-tenant business software: data leaking between customers, accounts protected only by a password, long-lived sessions, and permissions enforced only in the user interface.

100%	41	32	0
ACCOUNTS ON TWO-FACTOR	SECURITY CHECKS EXECUTED	ATTACKS BLOCKED	CROSS-CUSTOMER DATA EXPOSURE

What protects customers

LAYER	PROTECTION IN PLACE
Identity	Mandatory two-factor authentication (TOTP authenticator app) for every account, with no opt-out. Passwords hashed with argon2id. Accounts lock after repeated wrong passwords.
Sessions	15-minute access tokens held only in memory; rotating refresh tokens in secure, script-proof cookies; automatic detection of stolen tokens that signs the attacker out everywhere.
Access control	Every action checked on the server against a role permission matrix. Financial values are removed from responses for roles not entitled to them. Audit verification requires a second person.
Customer isolation	The customer an account belongs to is taken only from its cryptographically signed token, and every database query is structurally bound to that customer.
Data protection	Authenticator secrets encrypted with AES-256-GCM; all tokens and recovery codes stored only as one-way hashes; HTTPS enforced with HSTS.
Platform operations	Tecres staff use a completely separate identity system, console and signing key; their access is itself protected by two-factor authentication and logged.
Monitoring	Sign-in activity with IP addresses, automatic alerts for failed-login spikes, a full activity log per organization, and instant suspension and forced password reset.

Independent verification

The controls in this report were tested by attacking them directly: **34 automated attack scenarios** run against the real application and database, plus **7 live tests** against a running server, following OWASP methodology. Every attempt to read or change another customer's data, forge or tamper with a session token, skip two-factor authentication, escalate a role, or inject SQL **was blocked**. One vulnerability found during testing was fixed immediately and re-verified with a permanent regression test (section 11). Further hardening items identified by the assessment are tracked to closure in Tecres' internal remediation register.

ALL READERS

2 Security architecture at a glance

TecresTrackIT follows a **defense-in-depth** design: every request crosses several independent layers, and each one would still protect data if another failed. The web application is treated as untrusted; **all enforcement happens on the server.**

1 · Transport: HTTPS only, HSTS for one year, strict CORS allow-list

2 · Edge protection: security headers (CSP, anti-framing, no-sniff), request-size limits, rate limiting per IP and per account

3 · Authentication: signed, purpose-bound, 15-minute tokens; mandatory two-factor sign-in

4 · Organization context: the customer is taken from the verified token; suspended customers are refused on every request

5 · Authorization: role permission check on every action

6 · Input validation: every field checked against a strict schema; unknown fields discarded

7 · Data access: org-scoped query layer; ownership re-checked on direct lookups and file downloads

8 · Data at rest: argon2id passwords, AES-256-GCM secrets, hashed tokens

9 · Observability: activity and security logs, automated alerts, platform risk dashboard

Two identity systems, fully separated

Customer users and Tecres staff are kept apart at every level. They live in different database tables, have different roles, sign in at different addresses, and hold tokens signed with **different secret keys**. A customer's token is mathematically incapable of opening the Tecres console, and a Tecres token cannot act inside a customer's workspace.

Customer users Roles: super admin, admin, asset manager, staff	**Tecres platform admins** Roles: platform owner, platform support
Own signing key and token audience	Separate signing key and audience
Own server checks: authenticate, org context, authorize	Separate server checks and permissions
Own browser store and API client	Separate store, client and minimal console

Workspaces are provisioned, never self-registered

Customer workspaces are created by Tecres platform owners only, together with their first administrator. Further staff join by invitation from their own administrator. This keeps every organization on the platform a known, contracted customer.

SECURITY, ENGINEERING

3 Identity and authentication

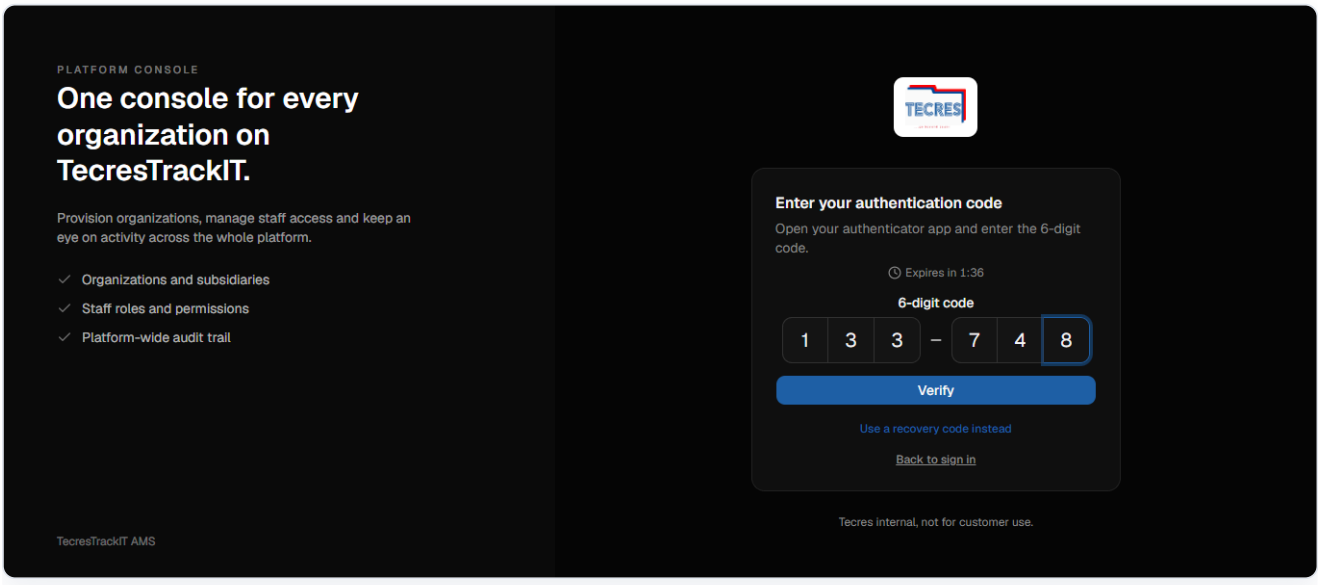
3.1 Password security

- Passwords are hashed with **argon2id**, the algorithm recommended by OWASP and winner of the Password Hashing Competition. It is deliberately slow and memory-hard, which makes offline guessing expensive even if a database were ever copied.
- Passwords are never stored, logged or returned in plain text, and every API response that includes a user removes the hash.
- Every sign-in failure (unknown email, wrong password, inactive or locked account) returns the identical message *"Invalid email or password"*, so the sign-in form does not reveal which part was wrong.

3.2 Mandatory two-factor authentication

Every account, customer and Tecres alike, must use an authenticator app (Google Authenticator, Microsoft Authenticator, 1Password and similar). There is **no setting to turn it off for an organization** and no way to reach a working session without it. A correct password earns only a short-lived, single-purpose token that can do nothing except be exchanged for a session when the 6-digit code is entered.





The second step of every sign-in. The code expires with the pre-auth token, shown by the countdown.

Standard	RFC 6238 time-based one-time passwords, 30-second codes
First sign-in	The user scans a QR code, confirms one code, and only then receives a session. Enrollment cannot be skipped.
Secret storage	Each user's authenticator secret is encrypted with AES-256-GCM before it reaches the database
Replay protection	Customer sign-ins record the last accepted time-step, so a code that has been used cannot be used again

Recovery	10 single-use recovery codes issued at enrollment, stored only as SHA-256 hashes

3.3 Brute-force and credential-stuffing defenses

CONTROL	SETTING	PROTECTS AGAINST
Account lockout	5 wrong passwords lock the account for 15 minutes, even if the next guess is correct	Password guessing against one account
Sign-in rate limit	10 attempts per 15 minutes per IP address and account	Automated guessing and credential stuffing
Sensitive actions	5 per 15 minutes per IP (authenticator setup, password reset)	Abuse of reset and enrollment
Global limit	600 requests per 15 minutes per IP	Scraping and flooding
Failed-sign-in trail	Every failure logged with IP; spikes raise an alert to Tecres	Undetected attacks

3.4 Password reset

- The reset form responds identically whether or not the email exists.
- Reset links are random 256-bit tokens, stored only as hashes, valid for 60 minutes and usable once.
- Completing a reset signs the account out of every device.

SECURITY, ENGINEERING

4 Session management

TOKEN	LIFETIME	WHERE IT IS KEPT	WHY IT IS SAFE
Access token (signed JWT)	15 minutes	Browser memory only	Never written to disk or local storage; expires quickly if exposed
Refresh token (random 256-bit)	30 days, rotated on every use	`httpOnly`, `Secure` cookie limited to the sign-in endpoints	Unreadable by JavaScript; stored on the server only as a hash

4.1 Purpose-bound tokens

Every token states what it is for (its *audience*), and every check accepts exactly one purpose. A token earned by the password step can therefore never be used as a session, a customer token can never be used on the Tecres console, and the reverse is equally impossible.

TOKEN PURPOSES · backend/src/lib/jwt.ts

```
const AUDIENCE = {
  access: "tecres:tenant:access",
  preAuth: "tecres:tenant:pre-auth",
  platformAccess: "tecres:platform:access",
  platformPreAuth: "tecres:platform:pre-auth",
} as const;

export function verifyAccessToken(token: string) {
  return jwt.verify(token, env.JWT_ACCESS_SECRET, { audience: AUDIENCE.access });
}
```

4.2 Rotation and stolen-token detection

Each time the application renews a session, the old refresh token is retired and a new one issued. If a retired token is ever presented again, it can only mean someone copied it. The platform then **revokes the entire session chain**, signing out both the attacker and the real user, who simply signs in again.

THEFT DETECTION · backend/src/modules/auth/auth.service.ts

```
if (existing.revokedAt) {
  // Reuse of a revoked token is a theft signal: revoke the entire rotation family.
  await db.update(refreshTokens).set({ revokedAt: new Date() })
    .where(and(eq(refreshTokens.familyId, existing.familyId), isNull(refreshTokens.revokedAt)));
  throw new UnauthorizedError("Refresh token reuse detected, session revoked");
}
```

4.3 Ending sessions

- **Sign out** revokes the current session; **sign out everywhere** revokes every session of the account.
- A password reset, or a reset forced by Tecres, revokes all sessions immediately.
- Suspending an organization blocks all of its users on their very next request.

SECURITY, ENGINEERING

5 Authorization and access control

Permissions are enforced by the server on every request through a single `authorize()` check tied to a central permission matrix. The web application hides buttons a role cannot use, purely for convenience; calling the API directly gives exactly the same answer as the screen.

ENFORCEMENT POINT · backend/src/middleware/authorize.ts

```
export function authorize(permission: keyof PermissionSet) {
  return (req, _res, next) => {
    if (!req.user) throw new UnauthorizedError();
    if (!hasPermission(req.user.role, permission)) throw new ForbiddenError(`Missing required permission: ${permission}`);
    next();
  };
}
```

5.1 Customer roles

CAPABILITY	SUPER ADMIN / ADMIN	ASSET MANAGER	STAFF
Organization profile, branding and settings	✓		
Create and edit assets	✓	✓	
Delete assets; correct a locked asset tag	✓		
Check assets out and in	✓	✓	
Request maintenance	✓	✓	✓
Update and complete maintenance	✓	✓	
Manage people and invitations	✓		
Manage vendors; export reports; print labels	✓	✓	
Assign and verify audits	✓	✓	
Carry out an audit assigned to them	✓	✓	✓
See purchase price, current and salvage value	✓	✓	

5.2 Financial data is removed, not hidden

For roles without valuation rights, monetary fields are **deleted from the server's response** before it leaves the API. They are not blanked in the browser; they are never sent. Inspecting network traffic reveals nothing more than the screen shows.

BACKEND/SRC/LIB/PERMISSIONS.TS

```
export const VALUATION_FIELDS = ["purchasePrice", "currentValue", "salvageValue", "totalValuation"] as const;

export function stripValuationFields(obj, role) {
  if (hasPermission(role, "canViewAssetValuation")) return obj;
  const clone = { ...obj };
  for (const field of VALUATION_FIELDS) delete clone[field];
  return clone;
}
```

5.3 Segregation of duties

Asset audits follow a controlled workflow (draft, assigned, in progress, submitted, verified). The person who verifies an audit can **never** be the person who submitted it, whatever their role. This is enforced in the server's audit service, and every status change is permanently recorded with who made it.

5.4 Tecres platform roles

CAPABILITY	PLATFORM OWNER	PLATFORM SUPPORT
View fleet dashboard, organizations, users, activity, alerts	✓	✓
Resolve alerts	✓	✓
Create and suspend organizations; adjust quotas	✓	
Force a user's password reset; manage platform admins	✓	

Tecres staff cannot sign in as a customer or act inside a customer's workspace. An earlier impersonation capability was deliberately removed from the product.

SECURITY, ENGINEERING

6 Tenant isolation

All customers share one database, so keeping each organization's data separate is the platform's most important property. Isolation is built into the structure of the code rather than left to careful coding.

6.1 Three rules, enforced structurally

1. **The organization comes only from the signed token.** No customer-facing endpoint accepts an organization ID from the request. A value smuggled into a request is discarded, and editing a token breaks its signature.
2. **Every query is bound to one organization.** Customer data is read and written through `scopedDb(orgId)`, whose every operation adds the organization filter. There is no function in it that omits the filter, so an unscoped query cannot be written by accident.
3. **Ownership is re-checked on direct access.** Looking up any record or file by its ID confirms it belongs to the caller's organization. A record from another organization answers "not found", so its existence is not even confirmed.

ORG-SCOPED DATA ACCESS · backend/src/lib/scopedDb.ts

```
export function scopedDb(orgId: string) {
  return {
    findById(table, id) {
      return db.select().from(table)
        .where(and(eq(table.orgId, orgId), eq(table.id, id)))   // the org filter is not optional
        .then((rows) => rows[0]);
    },
    insert(table, data) {
      return db.insert(table).values({ ...data, orgId }).returning();   // orgId always injected
    },
    // findMany, update, remove: same pattern
  };
}
```

FILE DOWNLOADS RE-CHECK OWNERSHIP · backend/src/modules/files/files.router.ts

```
const meta = await getFileMeta(req.params.id);
if (!meta) throw new NotFoundError("File not found");
if (meta.orgId !== req.user!.orgId) throw new NotFoundError("File not found");
```

6.2 Organization status on every request

Before any customer request is processed, the organization named in the token is loaded and checked. A suspended organization is refused immediately, even if its users hold tokens that have not yet expired.

> **VERIFIED BY ATTACK**
>
> Penetration tests tried to read, edit and delete another organization's assets by ID, download another organization's files, inject a foreign organization ID, swap the organization inside a token, and search across organizations. **Every attempt was blocked** (tests PT-03, PT-19, PT-20, PT-21, PT-28 and the core isolation test).

SECURITY, ENGINEERING

7 Data protection

7.1 Data at rest

DATA	PROTECTION
Passwords	argon2id one-way hash
Authenticator secrets	AES-256-GCM authenticated encryption, fresh random IV per secret, key held outside the database
Refresh, reset and invitation tokens	SHA-256 hash only; the usable value exists only in the user's cookie or email
Recovery codes	SHA-256 hash, invalidated on first use
Uploaded files	Stored per organization (Cloudinary, S3 or the database) with ownership checked on every download

ENCRYPTION OF AUTHENTICATOR SECRETS · backend/src/lib/crypto.ts

```
export function encryptSecret(plaintext: string): string {
  const iv = crypto.randomBytes(12);
  const cipher = crypto.createCipheriv("aes-256-gcm", ENCRYPTION_KEY, iv);
  const encrypted = Buffer.concat([cipher.update(plaintext, "utf8"), cipher.final()]);
  return [iv, cipher.getAuthTag(), encrypted].map((b) => b.toString("hex")).join(".");
}
```

7.2 Data in transit

- HTTPS is required; **HSTS** instructs browsers to refuse plain HTTP for one year, including subdomains.
- Session cookies carry `Secure`, `httpOnly` and a path restricted to the sign-in endpoints.
- Only the Tecres web application's own address may call the API from a browser (CORS allow-list).

7.3 Secrets management

Signing and encryption keys live in the server environment, never in source code. At start-up every key is validated: the server **refuses to run** if any is missing or shorter than 32 characters. Customer and Tecres tokens are signed with different keys.

7.4 Data minimization

- Financial values are withheld from roles that do not need them (section 5.2).
- The public sign-in page can learn only a workspace's name, logo and color, never its data.
- Expired tokens and invitations are deleted automatically every hour.

SECURITY, ENGINEERING

8 Application and API hardening

8.1 Strict input validation

Every request body, query string and URL parameter is checked against a Zod schema before any business logic runs. Values of the wrong type or format are rejected, and fields the schema does not name are discarded, which prevents mass-assignment attacks. Database access uses parameterized queries through the Drizzle ORM, and user-selectable sort columns map to a fixed list, so user input is never assembled into SQL text.

EXAMPLE SCHEMA · backend/src/modules/people/people.schema.ts

```
export const personUpdateSchema = z.object({
  name: z.string().min(1).max(255).optional(),
  role: z.enum(["super_admin", "admin", "asset_manager", "staff"]).optional(),
  isActive: z.boolean().optional(),
  // anything else in the request body is dropped
});
```

8.2 Security headers

HEADER	VALUE	EFFECT
`Content-Security-Policy`	`default-src 'self'; object-src 'none'; frame-ancestors 'self'` …	Blocks injected scripts and foreign content
`Strict-Transport-Security`	`max-age=31536000; includeSubDomains`	HTTPS only for one year
`X-Frame-Options`	`SAMEORIGIN`	Prevents clickjacking
`X-Content-Type-Options`	`nosniff`	Prevents content-type confusion
`Referrer-Policy`	`no-referrer`	No URL leakage to other sites
`Cross-Origin-Opener/Resource-Policy`	`same-origin`	Isolates the API from other sites
`X-Powered-By`	removed	No technology fingerprinting

8.3 Safe error handling

Clients receive only an error code and a plain message. Stack traces, file paths and database details are written to the server log and never returned. A record belonging to another organization is reported as "not found" rather than "forbidden".

8.4 Resource limits

Request bodies are capped at 2 MB, file uploads at 20 MB and logos at 5 MB (images only), alongside the rate limits in section 3.3.

8.5 Browser-side safeguards

- Access tokens are kept in memory only, never in `localStorage` or `sessionStorage`.
- The customer application and the Tecres console use separate stores and API clients, so a token can never flow from one to the other.
- Concurrent session renewals are merged into one, so the application never trips its own stolen-token detection.

ALL READERS

9 Monitoring, audit trail and response

9.1 What is recorded

LOG	CONTENTS
Organization activity log	Successful and failed sign-ins with IP address, lockouts, authenticator enrollment, and changes to assets, tags, people and settings
Audit status history	Every audit transition, who made it and when; append-only
Platform admin log	Every Tecres action: organization created, suspended or reactivated, quota changed, password reset forced, admin invited, alert resolved
Server log	Structured request log with request IDs; errors at elevated level

9.2 Detection

- **Login-risk view:** the Tecres console lists accounts in the middle of a failed-sign-in spike, the IP addresses involved, and links straight to the account.
- **Automatic alerts** every 15 minutes: failed-sign-in spikes, quota breaches, expired plans and inactive organizations.

9.3 Response capabilities

SITUATION	RESPONSE AVAILABLE	TAKES EFFECT
A user's account may be compromised	Tecres forces a password reset (all sessions revoked); or the user signs out everywhere	Immediately
An organization must be stopped	Tecres suspends the organization	On every user's next request
A session token was copied	Detected automatically on reuse; whole session chain revoked	Automatic
A lost phone	The user signs in with a one-time recovery code	Immediately

ENGINEERING

10 Secure development practices

- **Security by structure.** The safe way is the easy way: org-scoped data access, a shared CRUD factory that includes permission checks, typed errors, and validation middleware on every route.
- **Typed end to end.** TypeScript on both sides; changes must pass a strict type-check and production build before they are considered done.
- **Security regression tests.** Authentication, isolation, token handling and the audit workflow are covered by integration tests against a real database, and every penetration test in section 11 is kept as a repeatable test, so a fixed weakness cannot quietly return.
- **Mandatory testing for sensitive changes.** Any change to authentication, org-scoped queries, tokens or audits must run the full integration and security suites.
- **Dependency auditing** of both applications' production dependencies as part of security review.
- **No secrets in code.** Keys come only from the validated environment.

ALL READERS

11 Penetration testing

11.1 Approach

Date	26 September 2026
Type	White-box (full source access), combining code review, an automated attack suite and live black-box testing
Standards	OWASP API Security Top 10 (2023), OWASP ASVS 4.0.3 Level 2, OWASP Web Security Testing Guide
Target	The real Express application and a real PostgreSQL database (no mocks), plus a running server for live tests. Production was not attacked.
Scale	34 automated attack scenarios and 7 live tests

1. **Review.** Every security-relevant component was read to form attack hypotheses.
2. **Attack.** Each hypothesis became an automated test that attacks the running application exactly as an outsider or a malicious user would.
3. **Probe live.** Headers, CORS, rate limiting and error responses were tested over HTTP against a running server.
4. **Fix and prove.** Anything found was fixed, and the attack kept as a permanent regression test.

11.2 Authentication and sessions

ID	ATTACK ATTEMPTED	OWASP	RESULT
PT-01	Forge an administrator token with no signature (`alg: none`)	API2	Blocked
PT-02	Forge a token signed with a guessed key	API2	Blocked
PT-03	Edit a genuine token to point at another organization	API2	Blocked
PT-04	Reuse an expired token	API2	Blocked
PT-05	Use a customer token on the Tecres console, and the reverse	API5	Blocked
PT-06	Use the password-only token as a session to skip two-factor	API2	Found & fixed
PT-07	Tell a wrong password from an unknown email by the message	API2	Blocked
PT-08	Guess a password: 5 wrong attempts, then the right one	API2	Blocked
PT-09	Replay a used authenticator code	API2	Blocked
PT-10	Use one recovery code twice	API2	Blocked
PT-11	Replay a stolen, already-rotated refresh token	API2	Blocked
PT-12	Discover accounts through the password-reset form	API2	Blocked
PT-13	Reset a password with a forged link	API2	Blocked

PT-15	Open the Tecres console with a password alone	API2	Blocked
PT-16	Perform owner-only actions as platform support	API5	Blocked
PT-33	Read the refresh token from JavaScript or a response	API2	Blocked
PT-34	Call 18 protected endpoints with no token	API2	Blocked

11.3 Authorization and tenant isolation

ID	ATTACK ATTEMPTED	OWASP	RESULT
PT-19	Read, edit and delete another organization's asset by guessing its ID	API1	Blocked
PT-20	Download another organization's file by ID	API1	Blocked
PT-21	Write into another organization by injecting its ID	API3	Blocked
PT-22	As staff, call delete, role-change, report and settings endpoints directly	API5	Blocked
PT-23	As staff, extract asset values from list responses	API3	Blocked
PT-24	Keep working in a suspended organization with a live token	API5	Blocked
ISO	List and search across organizations (core isolation test)	API1	Blocked

11.4 Injection and HTTP hardening

ID	ATTACK ATTEMPTED	OWASP	RESULT
PT-28	SQL injection in search and filters (4 classic payloads)	API8	Blocked
PT-30	Provoke errors to leak stack traces or file paths	API8	Blocked
PT-31	Look for missing security headers or a framework banner	API8	Blocked
PT-32	Call the API from a foreign website (CORS)	API8	Blocked

11.5 Live tests against a running server

ID	TEST	OBSERVED	RESULT
LIVE-1	Inspect response headers	CSP, HSTS (1 year), X-Frame-Options, nosniff, no-referrer, COOP, CORP; no banner	Pass
LIVE-2	Request from `https://evil.example`	No cross-origin permission granted	Blocked
LIVE-3	CORS pre-flight from that origin	Origin not allowed; browser blocks the call	Blocked
LIVE-4	12 rapid customer sign-in attempts	Throttled after 10 (HTTP 429)	Blocked
LIVE-5	12 rapid Tecres console sign-in attempts	Throttled after 10 (HTTP 429)	Blocked

LIVE-7	Path traversal and injected IDs in the URL	Clean 404/401; nothing leaked	**Blocked**

11.6 Found and fixed during testing

Testing identified that the short-lived token issued after the password step was accepted by the Tecres console as if sign-in had completed, which could have let someone holding a Tecres staff password view console pages without the authenticator code. It was fixed during the engagement by binding every token to a single purpose (section 4.1), re-tested (the attack now returns HTTP 401 on every route), and kept as a permanent regression test (`tokenPurpose.security.test.ts`).

11.7 Conclusion

> **RESULT**
>
> With the fix described in 11.6 in place, every attempt to access another customer's data, forge or tamper with tokens, bypass two-factor authentication, escalate privileges or inject SQL was blocked. The penetration test confirms that TecresTrackIT's core protections (mandatory two-factor authentication, purpose-bound short-lived sessions with theft detection, server-side authorization and structural tenant isolation) hold up against direct attack. Additional hardening opportunities identified by the assessment are tracked to closure in Tecres' internal remediation register and are re-verified by the same test suite.

ENGINEERING

A Re-running the security tests

Every penetration test is kept as code in the backend repository and runs against a separate PostgreSQL test database (setup in the backend `README.md`, "Testing").

COMMANDS · run in asset-management-backend

```
# one-off: bring the test database schema up to date
DATABASE_URL=<test database url> pnpm db:migrate

# everything: integration tests and the security suites
pnpm test

# only the penetration suite
pnpm vitest run src/test/security
```

Penetration suite	`src/test/security/pentest.security.test.ts`
Two-factor bypass regression	`src/modules/auth/tokenPurpose.security.test.ts`
Core integration tests	`src/modules/auth/auth.integration.test.ts` (sign-in, token theft detection, isolation, value hiding)

EXAMPLE ATTACK TEST · PT-19, cross-organization access

```
it("PT-19 another org cannot read, edit, or delete an asset by guessing its ID (IDOR)", async () => {
  const orgA = await onboardOrg("pta");
  const orgB = await onboardOrg("ptb");
  const { assetId } = await seedAsset(orgA.accessToken);
  await request(app).get(`/api/assets/${assetId}`).set(bearer(orgB.accessToken)).expect(404);
  await request(app).put(`/api/assets/${assetId}`).set(bearer(orgB.accessToken)).send({ name: "pwned" }).expect(404);
  await request(app).delete(`/api/assets/${assetId}`).set(bearer(orgB.accessToken)).expect(404);
});
```

Live tests are plain HTTP requests against a running server, for example the sign-in throttling test:

```
for i in $(seq 1 12); do
  curl -s -o /dev/null -w "%{http_code} " -X POST localhost:4000/api/auth/login \
    -H 'content-type: application/json' -d "{\"email\":\"probe@nowhere.invalid\",\"password\":\"guess$i\"}"
done   # observed: ten 401s, then 429
```

ALL READERS

B Glossary

Tenant / organization	A customer using TecresTrackIT. All share one database, separated by organization.
Platform admin	Tecres staff member with access to the platform console.
Two-factor (TOTP)	A 6-digit code from an authenticator app, changing every 30 seconds, required in addition to the password.
Recovery code	A single-use backup code for when the authenticator app is unavailable.
JWT	JSON Web Token: a signed token the server issues and can verify.
Audience	The part of a token stating what it is for; each check accepts only one.
Refresh token	A cookie-held token used to renew a session without signing in again.
IDOR	Insecure direct object reference: reaching another's record by changing an ID.
RBAC	Role-based access control.
argon2id	A slow, memory-hard password-hashing algorithm.
AES-256-GCM	Authenticated encryption: data can be neither read nor altered without the key.
HSTS	A header telling browsers to use only HTTPS for the site.
CORS	Browser rules that stop other websites calling the API with a user's credentials.
OWASP	Open Worldwide Application Security Project, publisher of the standards used here.